

**УНИВЕРЗИТЕТ „СВ. КИРИЛ И МЕТОДИЈ“ ВО СКОПЈЕ**

Факултет за информатички науки и компјутерско инженерство (ФИНКИ)

Предмет: Континуирана интеграција и испорака

---

ЕЛАБОРАТ ЗА ПРОЕКТНА ЗАДАЧА

# **EduEval — Student Grading App with LLMs**

---

Студент: Христијан Мијалков

Датум: 20.08.2026

GitHub репозиториум: [github.com/HowlinMan24/Student-Grading-App-with-LLMs](https://github.com/HowlinMan24/Student-Grading-App-with-LLMs)

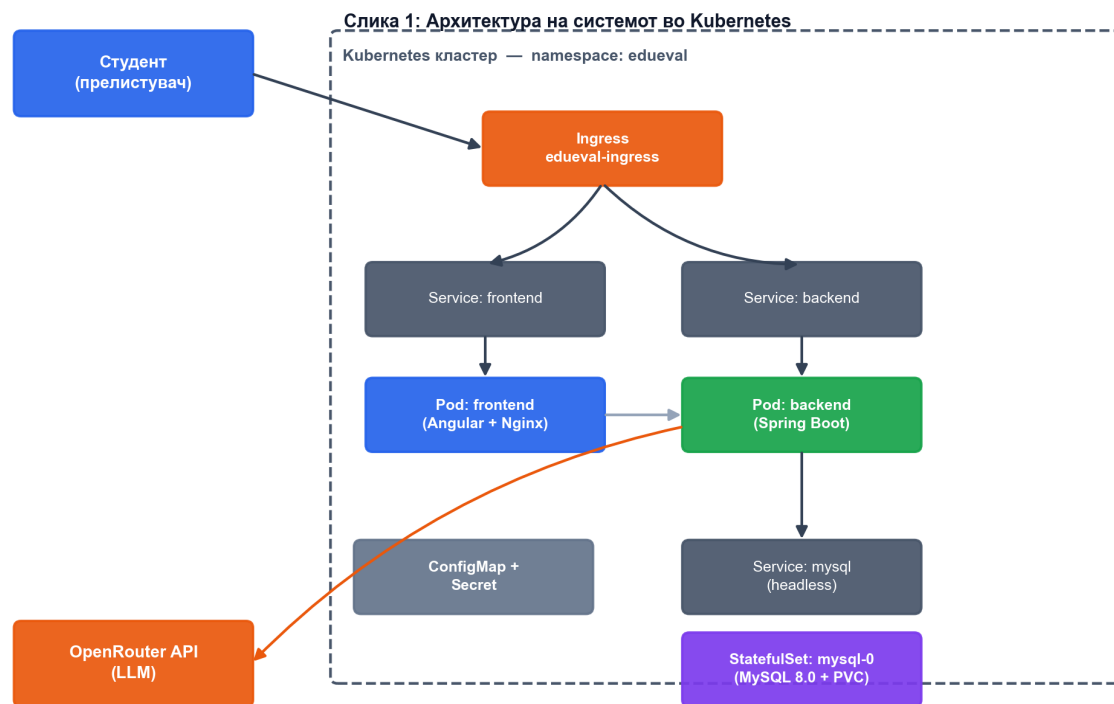
## 1. Вовед

EduEval (во интерфејсот именуван „EduBot“) е веб-апликација која на студентите им дава формативна повратна информација за нивните вежби, генерирана со голем јазичен модел (LLM). Студентот испраќа текстуален промпт и/или слика од својата работа, а бекендот го проследува барањето до LLM провајдер, кој одговара како едукативен асистент — проценува рубрика и дава повратна информација — но е системски инструктиран да одбие помош при активни тестови/испити.

Проектот е составен од три сервиси кои функционираат заедно: frontend (Angular 17, Tailwind CSS, служен преку Nginx), backend (Spring Boot, Spring Security + JWT) и база на податоци (MySQL 8.0). Целата апликација е докеризирана, оркестрирана локално со Docker Compose, автоматски градена и објавувана преку GitHub Actions CI/CD pipeline, и распоредена во посебен namespace во Kubernetes (Minikube).

## 2. Архитектура на системот

Секој сервис работи во сопствен Docker контејнер. Во Kubernetes, backend и frontend се распоредени како Deployment (без состојба, лесно се рестартираат), додека базата работи како StatefulSet со сопствен постојан волумен (PVC), бидејќи податоците мора да преживеат рестарт на подот. Единствената влезна точка однадвор е Ingress, кој патеката „/“ ја пренасочува кон frontend, а „/api“ директно кон backend.



Слика 1. Архитектура на распоредувањето во Kubernetes — namespace edueval: Ingress → Services → Pods, ConfigMap/Secret инјектирани како env променливи во backend-от, и врска до надворешниот OpenRouter API за самиот LLM повик.

### 3. Докеризација

Секој сервис има свој Dockerfile во коренот на репозиториумот:

- Dockerfile-backend — еднофазна изградба: eclipse-temurin:21-jdk-jammy база, ./mvnw package го гради runnable jar-от, стартува со java -jar.
- Dockerfile-frontend — двофазна (multi-stage) изградба: node:20 го гради Angular проектот (ng build), потоа само статичките фајлови се копираат во nginx:alpine, заедно со nginx.conf кој ги проследува /api барањата кон backend-от (само во Docker Compose контекст).

Двете images се објавени јавно на DockerHub, со tag :latest и tag со точниот commit SHA за секоја верзија:

- [hub.docker.com/r/howlinman/backend](https://hub.docker.com/r/howlinman/backend)
- [hub.docker.com/r/howlinman/frontend](https://hub.docker.com/r/howlinman/frontend)

### 4. Оркестрација со Docker Compose

docker-compose.yml ги подигнува трите сервиси (backend, frontend, db) на заедничка мостовна (bridge) мрежа. Базата има healthcheck (mysqladmin ping), а backend-от чека базата да биде „healthy“ пред да стартува (depends\_on: condition: service\_healthy). Податоците на MySQL се чуваат во именуван Docker volume, независен од животниот циклус на контејнерот.

Реална верификација (2026-09-19):

```
$ docker compose up --build -d
Container database Healthy
Container backend-container Started
Container frontend Started

$ curl -4 http://localhost:4200/api/auth/me → HTTP 401 (nginx исправно проследува /api до backend-от)
$ curl -4 http://localhost:8080/api/auth/me → HTTP 401 (backend директно, ист резултат)
```

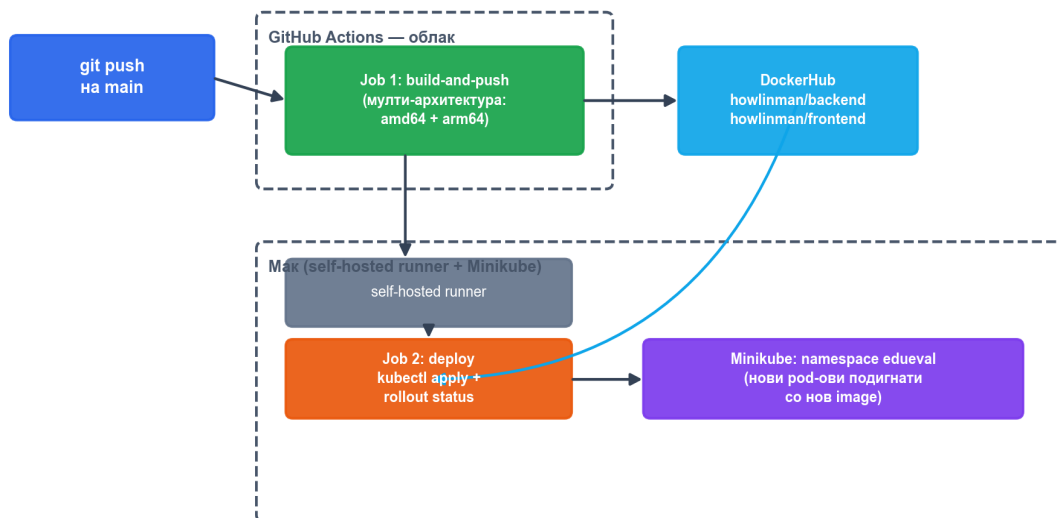
## 5. CI/CD со GitHub Actions

Избраната CI платформа е GitHub Actions (`.github/workflows/ci-cd.yml`), со целосно CI/CD решение (со бонус CD дел). Pipeline-от има две задачи:

- build-and-push (работи на GitHub-хостиран `ubuntu-latest` runner) — гради ги двете images за повеќе архитектури (`linux/amd64` и `linux/arm64`, преку QEMU + Buildx) и ги објавува на DockerHub, означени со `:latest` и `<git-sha>`.
- deploy (работи на self-hosted runner, регистриран на истиот Mac каде работи Minikube) — применува ги Kubernetes манифестите, ги ажурира Secrets од GitHub Actions Secrets, го „закотвува“ таговите на images од `:latest` на точниот commit SHA, и чека (`kubectl rollout status`) новите подови да станат подготвени.

Причината зошто deploy задачата работи на self-hosted runner, а не на GitHub-облак: GitHub-хостираните runner-и живеат во облак и физички не можат да дојдат до Kubernetes кластер кој постои само на localhost на овој Mac. Затоа овој компјутер е регистриран како GitHub Actions runner (`~/actions-runner`), инсталиран како позадински `launchd` сервис, секогаш подготвен да прифати нова задача.

Слика 2: CI/CD текот — од push до нов под



Слика 2. Целосниот CI/CD тек — од push на main до нов, распореден под во Minikube.

Успешно извршување на целиот pipeline (build + deploy): [GitHub Actions run #35427936753](#) — build 11m48s, deploy успешен, нови подови подигнати со најновиот commit.

## 6. Kubernetes манифести и распоредување

Сите Kubernetes манифести се во k8s/, поставени во посебен namespace edueval:

| Барање                         | Фајл                                           | Опис                                                              |
|--------------------------------|------------------------------------------------|-------------------------------------------------------------------|
| Deployment + ConfigMap/Secret  | backend-deployment.yaml                        | Spring Boot API, чита конфигурација и тајни преку env променливи  |
| Deployment                     | frontend-deployment.yaml                       | Angular SPA служен преку Nginx                                    |
| Service                        | backend-service.yaml,<br>frontend-service.yaml | Стабилен внатрешен endpoint (ClusterIP) за секој Deployment       |
| Ingress                        | ingress.yaml                                   | Единствена влезна точка однадвор;<br>/api → backend, / → frontend |
| StatefulSet + ConfigMap/Secret | mysql-statefulset.yaml                         | MySQL 8.0 со сопствен постојан волумен (PVC, 2Gi)                 |
| Namespace                      | namespace.yaml                                 | Изолиран namespace „edueval“ за сите ресурси на проектот          |

Демонстрација дека кластерот работи (kubectl, извршено локално):

```
$ kubectl get all -n edueval
pod/backend-75f55c94f9-jp52v 1/1 Running 0 51m
pod/frontend-66f99fc864-qfhh8 1/1 Running 0 51m
pod/mysql-0 1/1 Running 0 11h

deployment.apps/backend 1/1 1 1 10h
deployment.apps/frontend 1/1 1 1 10h
statefulset.apps/mysql 1/1 10h

service/backend ClusterIP 10.105.157.222 8080/TCP
service/frontend ClusterIP 10.104.54.59 80/TCP
service/mysql ClusterIP None 3306/TCP
```

Проверката на HTTP-рутирањето преку самиот Ingress (без порт-проследување) потврди дека „/“ ја служи фронтендот, а „/api/“ директно го достигнува backend-от: барање до заштитена рута без JWT токен коректно врати HTTP 401 (Unauthorized) — доказ дека Security-филтерот работи и во кластерот, не само локално.

## 7. Клучни проблеми решени при изработката

Целосниот дневник на проблеми (со точни логови и commit-и) е достапен во `DEVOPS_DOCUMENTATION.md` во репозиториумот. Накратко, најзначајните:

- Недостасувачки `nginx.conf`. Docker build постојано паѓаше бидејќи `frontend/nginx.conf` никогаш не беше commit-нат во git (беше означен како `untracked`). Решено со commit на фајлот.
- Хардкодирани тајни во јавен репозиториум. `backend/src/.../application.properties` привремено содржеше вистински лозинки и JWT `clean tekst` во локалната работна копија. Вратено на читање преку `environment` променливи пред commit, за да не се компромитираат тајните во јавниот репозиториум.
- CD задачата не можеше да го достигне Minikube. GitHub-облак runner нема мрежен пат до кластер на `localhost`. Решено со регистрирање `self-hosted runner` на истиот Mac каде работи Minikube.
- Несовпаѓање на архитектурата (`arm64` vs `amd64`). Images градени само за `amd64` не можеа да се повлечат на Apple Silicon Mac („no matching manifest for linux/arm64/v8“). Решено со мулти-архитектурна изградба (`QEMU` + `Buildx`).
- Застарена лозинка во MySQL volume. Контејнерот на базата беше стар 3 месеци и никогаш повторно иницијализиран, па задржа стара `root`-лозинка која веќе не се совпаѓаше со `.env` — `backend`-от постојано паѓаше со „Access denied“. Решено со `docker compose down -v` (бришење на волуменот) и повторно иницијализирање.

## 8. Линкови

- Репозиториум: [github.com/HowlinMan24/Student-Grading-App-with-LLMs](https://github.com/HowlinMan24/Student-Grading-App-with-LLMs)
- CI/CD pipeline: [.../actions](https://github.com/HowlinMan24/Student-Grading-App-with-LLMs/actions)
- Docker image (backend): [hub.docker.com/r/howlinman/backend](https://hub.docker.com/r/howlinman/backend)
- Docker image (frontend): [hub.docker.com/r/howlinman/frontend](https://hub.docker.com/r/howlinman/frontend)
- Целосна техничка документација: `DEVOPS_DOCUMENTATION.md` (во репозиториумот)
- Архитектурен документ со дијаграми на поврзаност: `ARCHITECTURE.md` (во репозиториумот)

## 9. Заклучок

Проектот EduEval целосно ги исполнува бараните точки: апликацијата е поставена на јавен git репозиториум, докеризирана, оркестрирана со Docker Compose, автоматски градена и објавувана преку GitHub Actions CI/CD pipeline (со дополнителен CD дел за бонус поени), и распоредена во посебен namespace во Kubernetes со Deployment, Service, Ingress и StatefulSet, сите со сопствени ConfigMap/Secret. Работата беше проверена не само со читање на конфигурацијата, туку со реално извршување, идентификување и решавање на неколку конкретни проблеми (документирани со вистински логови), и со верификација дека целиот систем работи од крај до крај — од барање во прелистувачот, преку Kubernetes Ingress, до одговор генериран од LLM.